# Artificial Intelligence

*Vegliante Amerigo*

---

Prof. Sperduti Alessandro

2026/2027

# Index

1. Agents . . . . . 1
    1.1. Types of Agents & Environments . . . . . 3
2. Problem Solving . . . . . 6

# 1. Agents

- **Agent**: An agent is a perceiving and acting entity, in our case we study **rational agents** which is an agent that:
  - Achieve its goals as much as possible given the avaiable information.
  - Searches to maximize the satisfaction of its needs by exploiting avaiable or acquirable information.

  If we abstract its definition an agent is a *function* of all the possible sequences of perceptions $P^*$ to (eligible) actions $A$ .

$$f : P^* \longrightarrow A$$

  For obvious reasons, among all the classes of environments and tasks, we look for the single agent (or class of agents) that has the best performances.

  - **Note**: For computational limitations *perfect rationality* is unachievable, so our job here is to design the best program given the resources we have at hand.

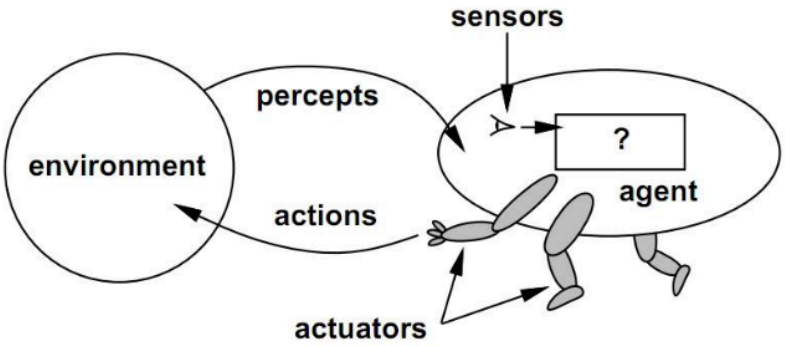



Figure 1: Structure of an agent

The **agent function** maps from all the possible sequences of perceptions $P^*$ to actions $A$ .

- **EXAMPLE**: Vacuum Cleaner Agent.
  - *Perceptions*: $\{\mathrm{A}, \mathrm{B}\}$
  - *Actions*: $\{\mathrm{Left}, \mathrm{Right}, \mathrm{Suck}, \mathrm{NoOp}\}$
  - *Status*: $\{\mathrm{Clean}, \mathrm{Dirty}\}$

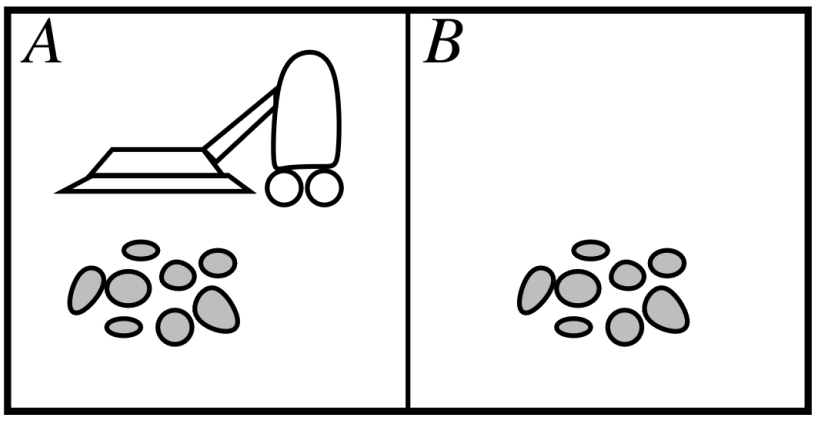



Figure 2: Vacuum cleaner agent example

| Percept Sequence | Action |
|---|---|
| [A, Clean] | Right |
| [A, Dirty] | Suck |
| [B, Dirty] | Left |
| ... | ... |

Table 1: Vacuum cleaner agent example

```
def reflex_vacuum_agent(location, status):
  if status == "Dirty":
    return "Suck"
  elif location == "A":
    return "Right"
  elif location == "B":
    return "Left"
```

- **Rationality**: It's a measure of performances set for an agent that evaluates the sequence of perceptions, it can be different based on the agent goalas and its context.
  A **rational agent** chooses any action that maximizes the expected value of the performance measure based on the sequence of perceptions obtained up to the current instant.
  - Rationality $\Longrightarrow$ Exploration, Learning, Autonomy.
- **Performance Environment Actuators Sensors (PEAS)**: It's a framework that we use to describe and evaluate rational agents because it defines how an agent interact with its environment.
  We can use it to design agents that can reach their goals effectively.
  - **Performance Measure**: It defines how the success of an agent is evaluated, determines wether the agent's actions are achieving the desired goal and guides the agent into making better decisions.
  - **Environment**: Everything surrounding an agent that can be observed or interacted with by the agent while performing its task. It can change overtime.
  - **Actuators**: They are the components that the agent uses uses to perform actions and interact with the evironment. They convert decisions into actions.
  - **Sensors**: They are the components that collect informations from the environment and provide them to the agent in order for it to make appropriate decisions.

After all this we can say that agents can be summarized with this simple formula:

$$\text{Agent} = \text{Architecture} + \text{Program}$$

Where:

- **Architecture**: It's the physical infrastructure supporting the execution of the program.
- **Program**: Is the entity that take the current percept as input from the sensors and return an action to the actuators.

```
percepts = [...]                    # A sequence, initially empty.
table = [...]                       # Table of actions, indexed by percept sequences.
def table_driven_agent(percept):
  percepts.append(percept)
  action = lookup(percepts, table)
  return action
```

## 1.1. Types of Agents & Environments

- **Simple Reflex Agents**: They are simple agent designs that are built on the concept of making choices based on immediate enviromental stimuli.
  - They are based on a set of predefined rules or conditional action pairs that dictate how the agents responds to unique sensory inputs.
  - They have a minimalist approach to decision making in order to prioritize simplicity and efficiency.
  - They don't have internal memory meaning they have only access to the present sensory input and base the action entirely on that.
  - **PROBLEM**: If the environment is *partially observable* it can fail, to avoid that it needs to track past perceptions (internal state).

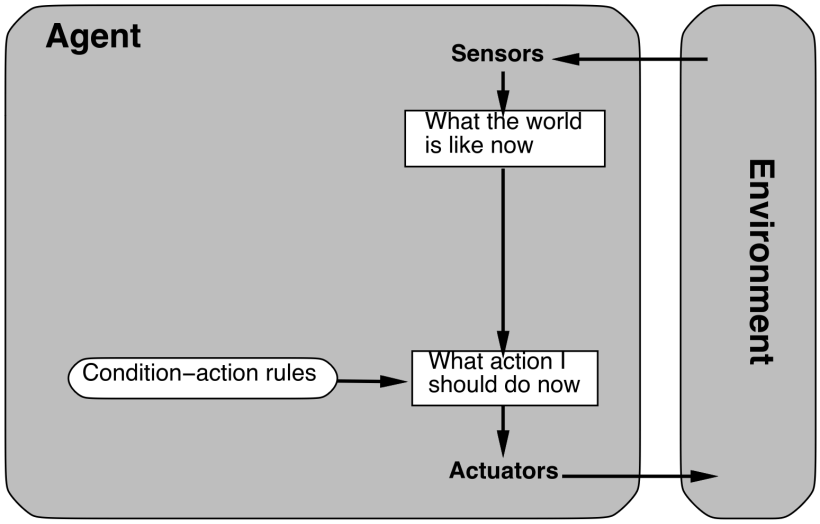



Figure 3: Simple Reflex Agent Schema

- **Model-Based Reflex Agents**: They are a more "complex" version of Simple Reflex Agents which, as we said before, are agents that only have the context of the current perception, Model-based Reflex Agents also mantain an internal representation (called **model**) of the environment which allows them to anticipate the consequences of their actions.

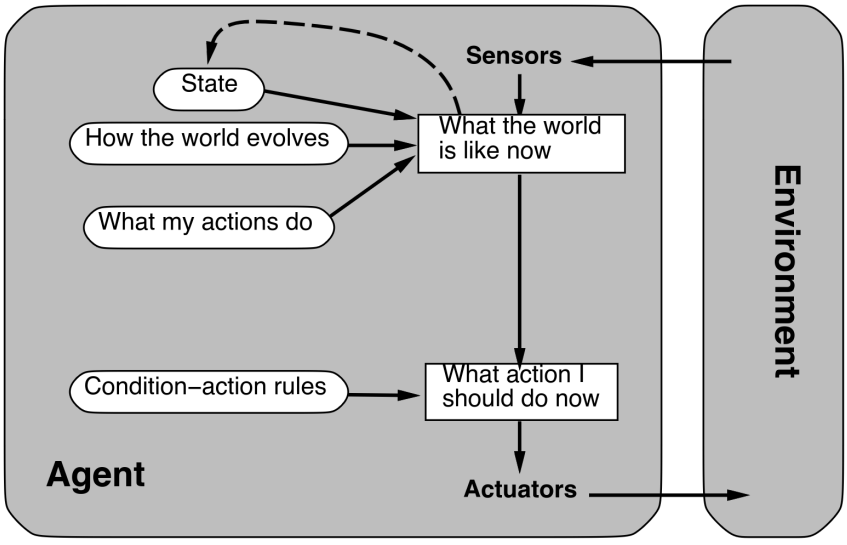



Figure 4: Model-based Reflex Agent Schema

- **Goal-based Agents**: They are agents designet to plan, execute, and adjust their actions dynamically to meet predefined goals. It's a useful approach in enviroments where adaptability is fundamental.
  - **Goal**: It's a specific objective for the agent to achieve, the goal itself also provides clear directions for the agent's decisions and actions.

This type of agent has three different phases:

- **Planning**: It's the phase of determining the sequence of actions required to achieve the goal.
- **Execution**: It's the phase where the agent actually execute the planned actions.
- **Adaptation**: During execution the agent might encounter obstacles or changes, this phase is required in order for the agent to modify its plans and actions dynamically in order to still achieve its goal.

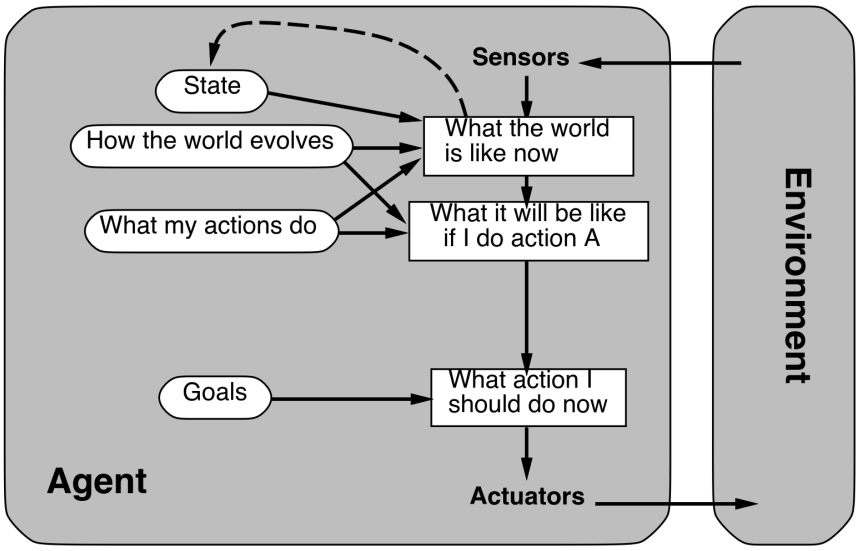



Figure 5: Goal-based Agent Schema

- **Utility-based Agents**: They are agents that have the ability to make rational decisions based on a utility function and are designed to optimize their performance by maximizing utility measures.
  - **Utility Function**: A utility function $U(s)$ maps each state $s$ to real numbers that indicate the utility of the state. Its purpose is to quantify the agent's preferences so that it can evaluate different states and since this agent's objective is to maximize utility it will choose actions that lead to the most desirable outcomes.

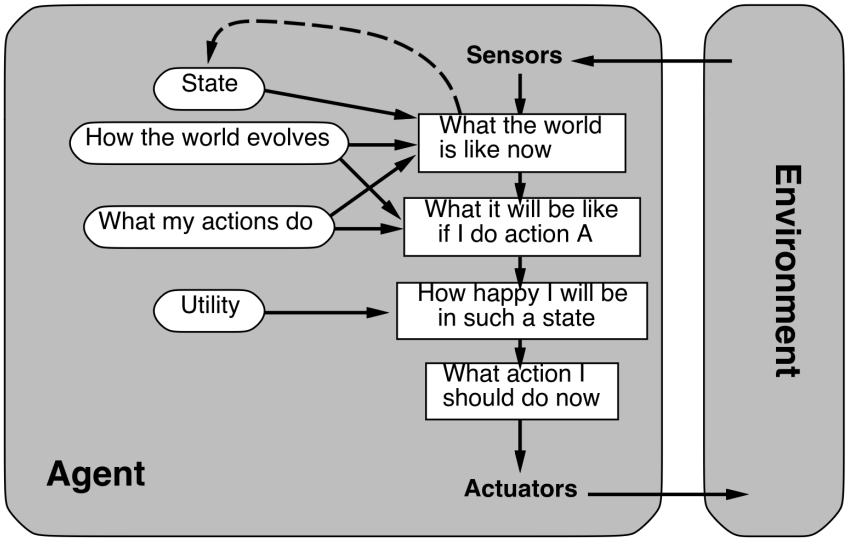



Figure 6: Utility-based agent schema

- **Learning Agents**: It's an entity designed to autonomously interact with its environment, it acquires informations from it and adapt its behaviour to improve performances over time. They can dynamicaly change their decision-making processes depending on experience instead of previously determined instructions.

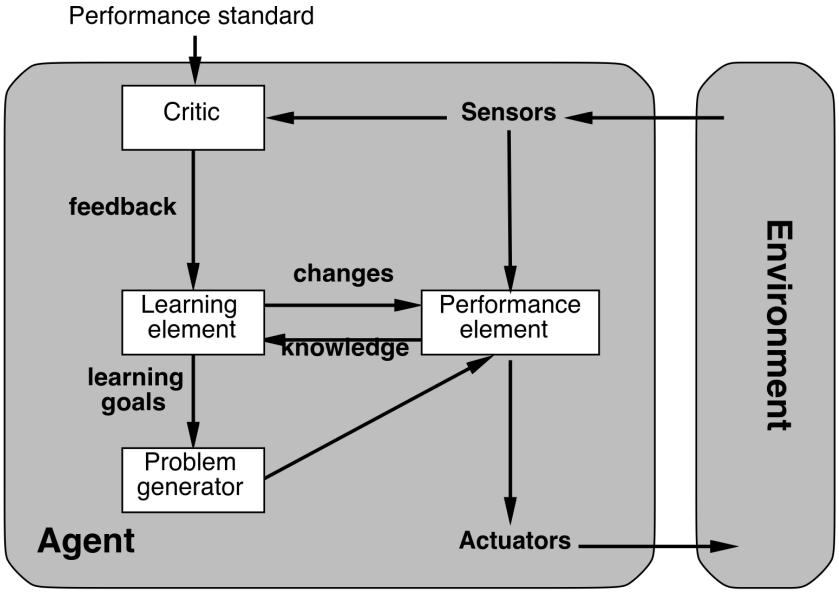



Figure 7: Learning agent schema

We can represent the environment in many ways:

- **Atomic Representation**: Each state of the environment is indivisible.
- **Factored Representation**: Splits up each state into a fixed set of variables that can have a value.
- **Structured Representation**: Objects and their relationships with each other can be described explicitly.

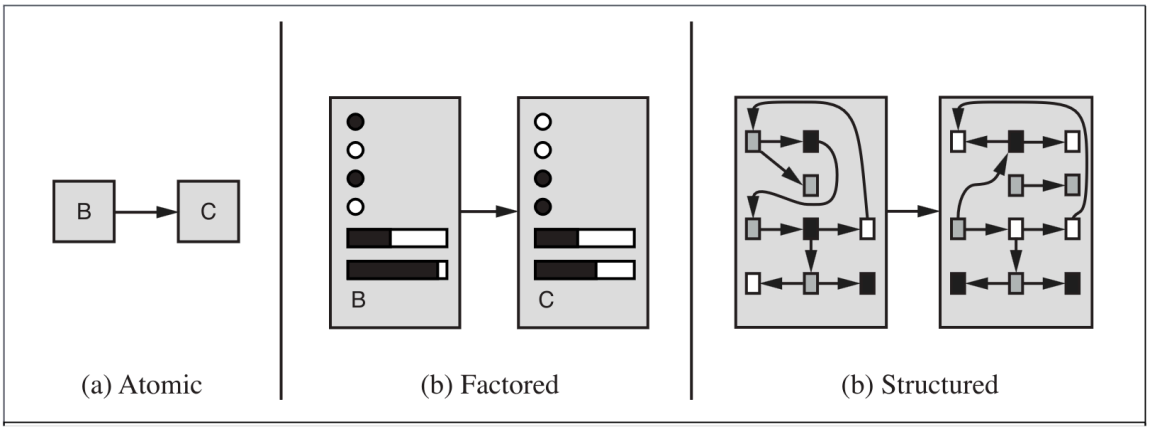



Figure 8: Enviroment representations

# 2. Problem Solving

In this section we are going to see **Uninformed Search Strategies**.
*How do we solve problems by searching?*

```
seq = []                              # An action sequence, initially empty.
state = State()                       # Description of the current "world" state.
goal = None                           # The goal, being a goal-based agent, initially empty.
problem = Problem()                   # A Problem formulation.

def simple_problem_solving_agent(percept):
  stat = UpdateState(state, percept)
  if seq.isEmpty() == True:
    goal = FormulateGoal(state)
    problem = FormulateProblem(state, goal)
    seq = Search(goal)
    if seq == "Failure":
      return None
  action = First(seq)
  seq = Rest(seq)
  return action
```

- **EXAMPLE**: We are on holiday in Romania, we are currently in Arad, tomorrow we have to catch a flight in Bucharest.
  - **Goal Formulation**: Be in Bucharest.
  - **Problem Formulation**:
    - *States*: The various romanian cities.
    - *Actions*: The drive between cities.
  - **Solution Structure**: Sequence of cities crossed from Arad to Bucharest.

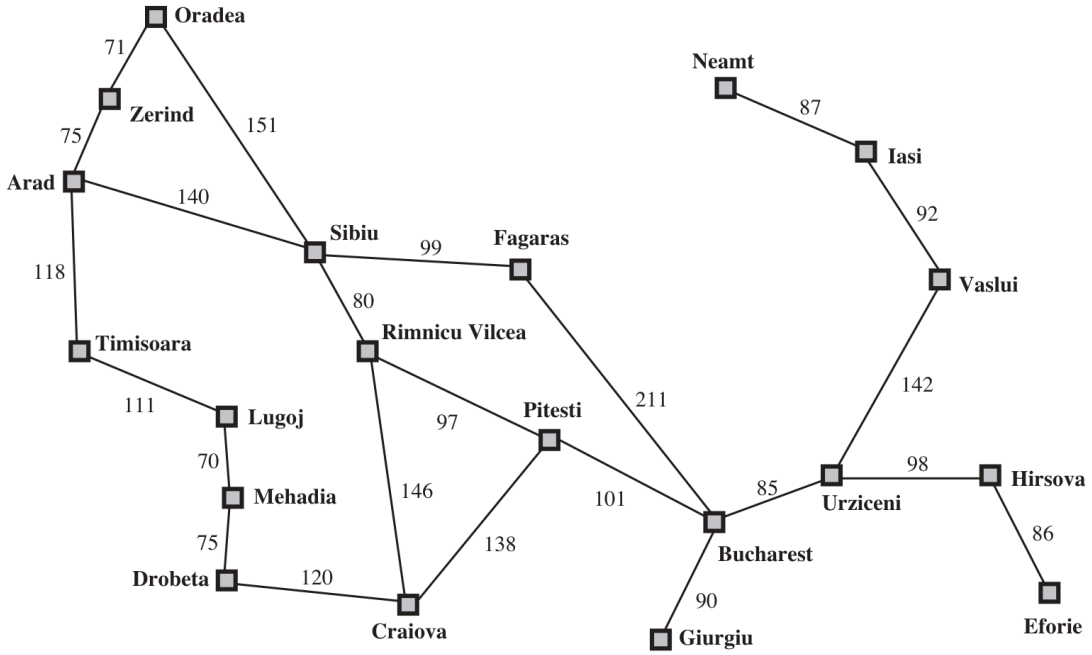



Figure 9: Simple graph of states and actions for the romania driving example

*How do we formulate this problem formally?*

- **Problem Formulation**: A problem is defined by four items:
  - **Initial State**: The starting point from which the agent starts to work towards its goal. In this case it will be “Arad”.
  - **Successor Function**: it’s a function

$$E(x) : \text{Actions} \longrightarrow \text{States}$$

In our case it would look like something like this

$$E(\text{Arad}) = \{(\text{Arad} \longrightarrow \text{Zerind}, \text{Zerind}), ...\}$$

Alternatively we can define separately:

- Actions that can be performed in a given state.
- The effect obtained by applyig each action.

- **Goal Test**
  - **Explicit**: $x = \text{at Bucharest}$
  - **Implicit**: $\text{NoDirt}(x)$
- **Path Cost**: Something that gets added up along the computation. In our case it could be *sum of distances, number of actions executed, ...* .
  - **Step Cost**: Is defined as

    $$c(x, a, y) \geq 0$$

    where:

    - $x$ is the *current state*.
    - $a$ is the *chosen action*.
    - $y$ is the *resulting state*.
- **Solution**: It's a sequence of actions that leads from the *initial state* to the *goal state*.